System Identification I: Markov Parameters from Data

Learning a system's impulse response directly from input/output data.

Everything so far assumed we already had a model. **System identification** goes the other way: given only measured inputs u_k and outputs y_k, recover a model. The most basic building blocks are the **Markov parameters** – the discrete impulse-response coefficients

$$h_0 = D, \qquad h_i = CA^{i-1}B \quad (i \geq 1).$$

In this tutorial you will:

- see that the output is a **convolution** of the input with the Markov parameters,
- stack that convolution into a linear least-squares problem, and
- recover the h_i from noisy data and rebuild the response.

Run with `publish('Intro.m')`, or step through with **Ctrl+Enter**.

Contents

A known system to generate data from

We use a known plant only to **make** data; the identification below pretends it can see just u_k and y_k.

```
A = [0.6 0.3; -0.2 0.5];
B = [1; 0.5];
C = [1 -1];
D = 0;
sys = ss(A,B,C,D,-1);           % -1 = discrete, unspecified sample time
n  = size(A,1);
```

The data record

Drive the system with a random input and collect a noisy output. In a real experiment this is all you would have.

```
rng(0);
N = 300;
u = randn(1,N);
y = lsim(sys, u).' + 0.01*randn(1,N);    % 1% measurement noise
```

Output = input convolved with the Markov parameters

For a causal system started at rest,

$$y_k = \sum_{i=0}^{p} h_i \, u_{k-i}.$$

Stacking this equation over $k = p+1, \ldots, N$ gives a linear system $y = \Phi\, h$ whose unknown is the vector of Markov parameters h. **Notice that** every row of Φ is just a window of past inputs.

```
p   = 15;                        % number of Markov parameters to estimate
Phi = zeros(N-p, p+1);
for k = 1:N-p
    Phi(k,:) = u(p+k:-1:k);      % [u_k, u_{k-1}, ..., u_{k-p}]
end
yv    = y(p+1:N).';
h_hat = Phi \ yv;                % least-squares solve
```

Compare with the truth

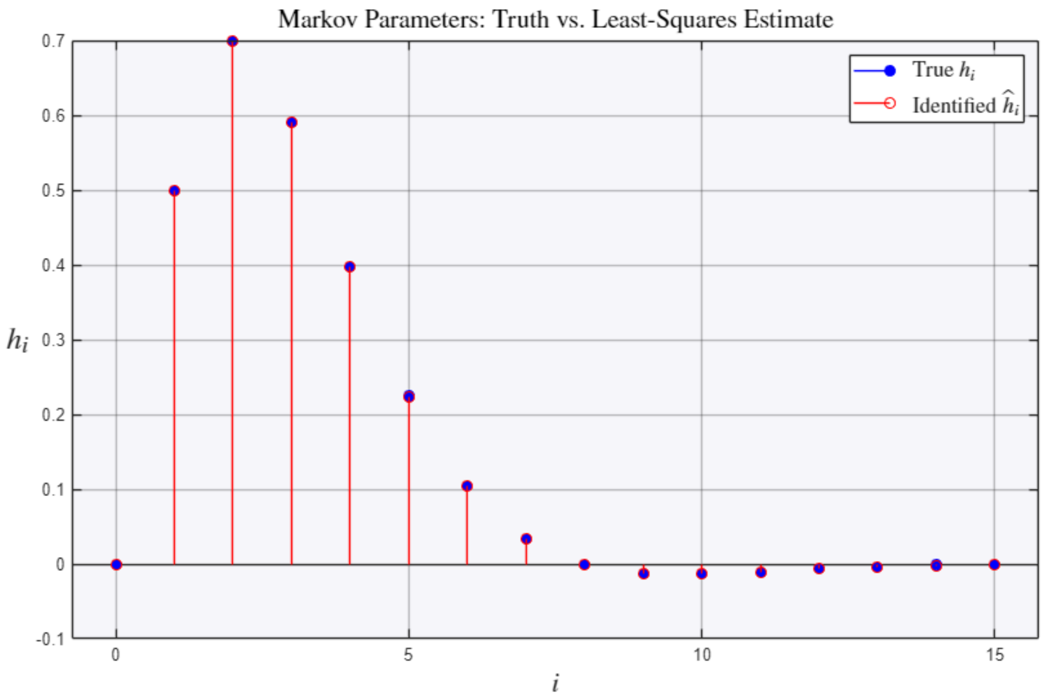
Because we built the data from a known system, we can check the estimate against the exact Markov parameters $h_i = CA^{i-1}B$.

```
h_true = zeros(p+1,1);
h_true(1) = D;
for i = 1:p
    h_true(i+1) = C*A^(i-1)*B;
end

figure
stem(0:p, h_true, 'b', 'filled')
hold on
stem(0:p, h_hat, 'r')
hold off
grid on
legend('True $h_i$','Identified $\hat{h}_i$','Interpreter','latex','FontSize',13)
title('Markov Parameters: Truth vs. Least-Squares Estimate','Interpreter','latex','FontSize',15)
ylabel('$h_i$','Interpreter','latex','FontSize',18); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$i$','Interpreter','latex','FontSize',16)

fprintf('Relative error in the identified Markov parameters: %.4f\n', ...
    norm(h_hat - h_true)/norm(h_true))
```

```
Relative error in the identified Markov parameters: 0.0016
```

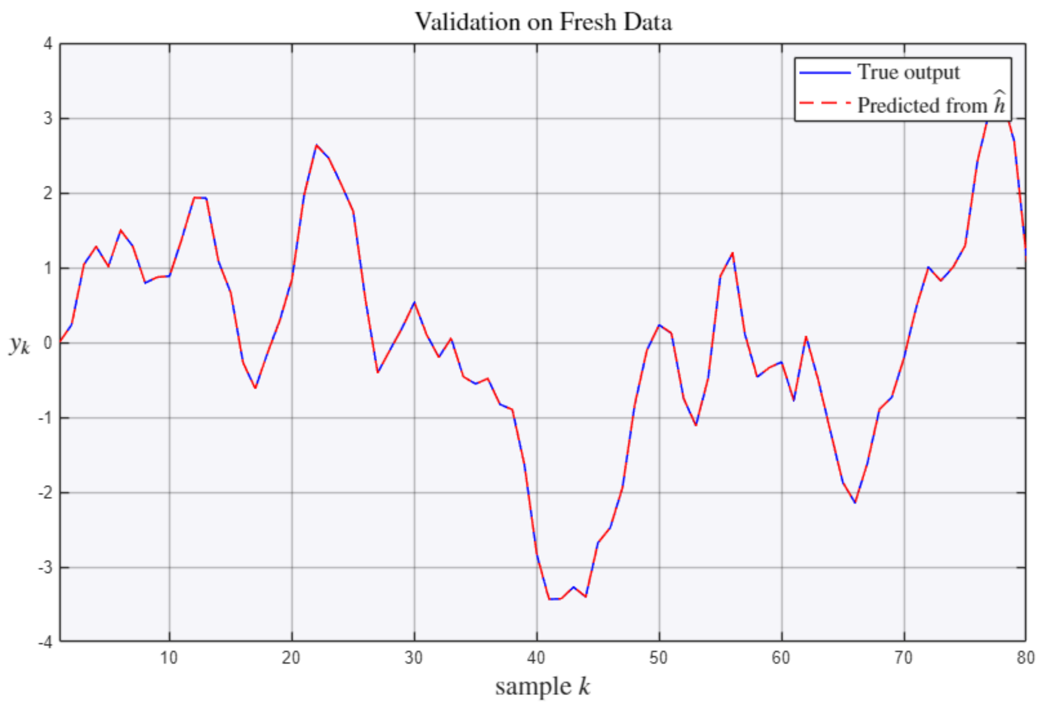


The identified model predicts new data

The real test of an identified model is **prediction**: convolve a fresh input with the estimated $\hat{h}_i$ and see whether it reproduces the true output.

```
u_test = randn(1,N);
y_true = lsim(sys, u_test).';
y_pred = filter(h_hat, 1, u_test);      % convolution with the FIR estimate

figure
plot(1:N, y_true,'b', 1:N, y_pred,'r--','LineWidth',1.1)
grid on; xlim([1 80])
legend('True output','Predicted from $\hat{h}$','Interpreter','latex','FontSize',13)
title('Validation on Fresh Data','Interpreter','latex','FontSize',15)
ylabel('$y_k$','Interpreter','latex','FontSize',16); set(get(gca,'YLabel'),'Rotation',0)
xlabel('sample $k$','Interpreter','latex','FontSize',16)
```



Try it yourself

- Raise the noise from `0.01` to `0.1` and watch the estimate degrade – then lengthen the data record `N` and see the least-squares fit recover.
- Shorten `p` below the settling length of the impulse response and notice the prediction lose accuracy (you truncated real dynamics).

Next: real systems rarely start at rest and can be lightly damped, so a plain impulse-response fit needs many terms. `OKID.m` introduces an **observer** into the estimator to fix both problems.

Summary

- The Markov parameters are the discrete impulse response, $h_i = CA^{i-1}B$.
- Output is their convolution with the input, which stacks into a linear least-squares problem $y = \Phi h$.
- Validate by **prediction** on data the fit never saw.